ALUMNICONNECT

Workflow and State Diagrams

Request, booking, allocation and session lifecycle - with triggers, stored procedures, transactions, constraints and notification processing

Document type DBMS Project Workflow Specification

Project AlumniConnect - Alumni-Student Mentorship Platform

Version 1.0

Date 2 November 2026

Database baseline MySQL 8.x with InnoDB; Redis and Neo4j for supporting workloads

Purpose Translate the project blueprint into explicit lifecycle rules and implementation guidance

Prepared from the supplied Alumni-Student Mentorship Platform: DBMS Project Blueprint and the supplied DBMS Unit 2 and Unit 4 course materials.

01 / ORIENTATION

Document guide and modelling conventions

This document specifies the operational lifecycle of AlumniConnect. It complements the Product Requirements Document and Conceptual ER Diagram; it does not replace the logical schema or final application API contract.

Scope covered

- Student discovers a mentor and submits a mentorship request.
- Mentor accepts or rejects; acceptance creates a mentor allocation in the same SQL transaction.
- Mentor publishes availability; student books an eligible slot through an atomic operation.
- A planned booking is distinguished from the actual offline mentorship/mock interview session.
- Participants submit session feedback; a student may submit a mentor review after a completed session.
- Business events are stored in a transactional outbox and delivered asynchronously.
- MySQL triggers and procedures protect transitions, record audit history, and centralise critical database operations.

How to read the diagrams

Notation	Meaning
Start dot	The lifecycle begins or the initiating action occurs.
Rounded state box	A persistent state stored in the database.
Arrow label	The event, actor action, or guard condition that permits transition.
Terminal / double-ring endpoint	A completed path; the original business record remains stored.
Dashed connector / note	Supporting operation, external delivery or a proposed policy.
Guard	A condition that must be true before a state change is accepted.

Requirement provenance

Label	Interpretation
Blueprint-defined	Explicitly stated by your project blueprint. These are preserved.
Course concept	A DBMS technique covered in the supplied Unit 2 or Unit 4 notes.
Recommended design	A practical implementation choice added to make the workflow testable. Team confirmation may be needed.
Open decision	The source does not specify the policy; decide it before coding.

Important scope boundary

The blueprint uses a modular monolith. MySQL is the authoritative source of truth for identity, requests, allocations, availability, bookings, sessions, feedback and audit data. Redis is a cache/temporary-state layer; Neo4j is a graph projection for discovery. Neither Redis nor Neo4j should approve or commit a booking.

02 / OVERALL LIFECYCLE

End-to-end workflow architecture

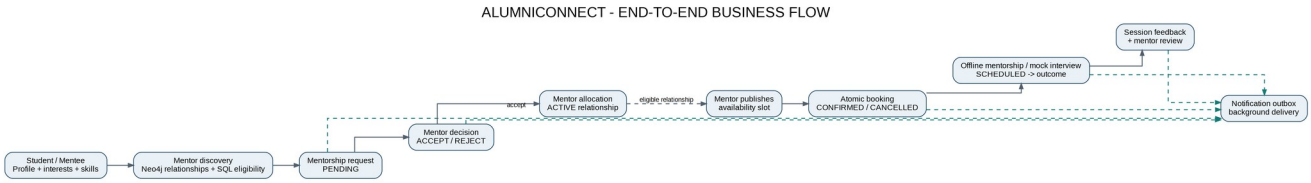


Figure 1. Business flow from discovery to session feedback; dashed paths represent notification event creation.

Lifecycle ownership by component

Component	Owns / decides	Must not decide
Frontend portals	Collect user actions; display current state and allowed actions.	Bypass server-side authorization or treat a cached state as authoritative.
FastAPI modular backend	Authenticate actor, authorise action, validate request context, call database routines.	Make a slot appear booked without a committed MySQL transaction.
MySQL / InnoDB	Canonical records, state transitions, foreign keys, unique constraints, transactions, audit and outbox.	Send email or call external providers inside a trigger.
Redis	Cache mentor cards, skills/domains, recommendations; temporary rate-limit/session state.	Own allocation, availability or booking truth.
Neo4j	Relationship-oriented mentor discovery and recommendation paths.	Override MySQL eligibility, capacity or current booking status.
Outbox worker	Deliver notification events and retry failures.	Invent an event not committed by the business transaction.

Key separation of concerns

A request is a proposal to be mentored. An allocation is the accepted mentorship relationship. A booking is a planned appointment against one availability slot. A session is the actual meeting and its outcome. Feedback and mentor review are separate records. Keeping these concepts separate prevents a planned appointment from being mistaken for a completed mentoring interaction.

03 / REQUEST STATE DIAGRAM

Mentorship request lifecycle

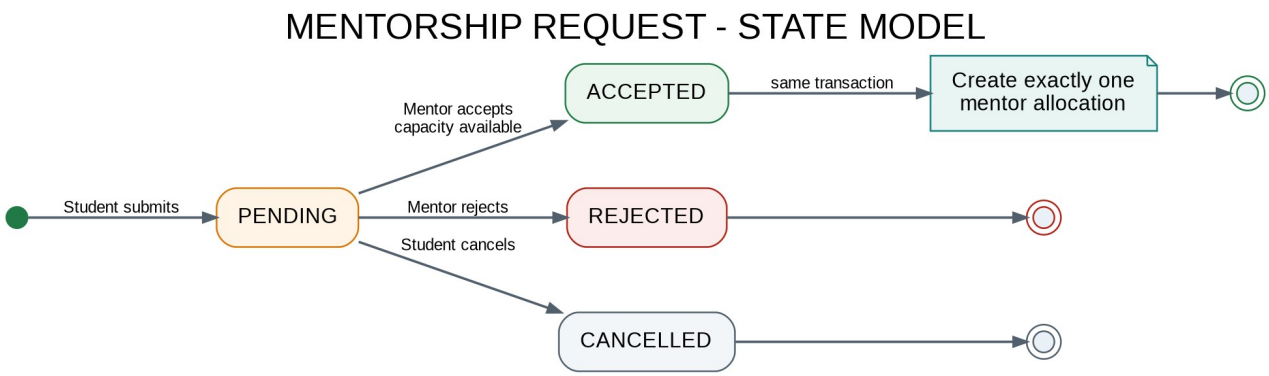


Figure 2. Blueprint-defined request states: PENDING, ACCEPTED, REJECTED and CANCELLED.

State transition catalogue

Current state	Event / actor	Guard conditions	Next state	Database action
Not yet submitted	Student submits request	Student profile active; target mentor exists and is discoverable; requester is not the same person as mentor (recommended).	PENDING	Insert request; insert request-submitted outbox event.
PENDING	Mentor accepts	Actor is the assigned mentor; request still pending; mentor is eligible and has remaining capacity.	ACCEPTED	Create one allocation, update request, append outbox event; same transaction.
PENDING	Mentor rejects	Actor is the assigned mentor; request still pending.	REJECTED	Update request; store decision timestamp/reason; enqueue notification.
PENDING	Student cancels	Actor owns the request; request still pending.	CANCELLED	Update request; retain history; enqueue cancellation event if required.
ACCEPTED / REJECTED / CANCELLED	Any later response	No normal transition is allowed. Corrections require an audited administrator process (recommended).	No change	Reject invalid transition.

Business rules

- A submitted request enters PENDING. The blueprint explicitly lists the resulting states as Accepted, Rejected or Cancelled.
- An accepted request can create at most one allocation.
- Rejected and cancelled requests remain in history; do not delete them to represent a decision.
- The mentor accepting the request must be the mentor to whom the request was addressed.
- The acceptance operation must check capacity and prevent duplicate active allocations.

Design note

The blueprint does not define a Draft state, expiry period, resubmission policy or a student withdrawal path after acceptance. Do not silently add these states. Record them as product decisions if the team wants them.

04 / ALLOCATION STATE MODEL

Mentor allocation lifecycle

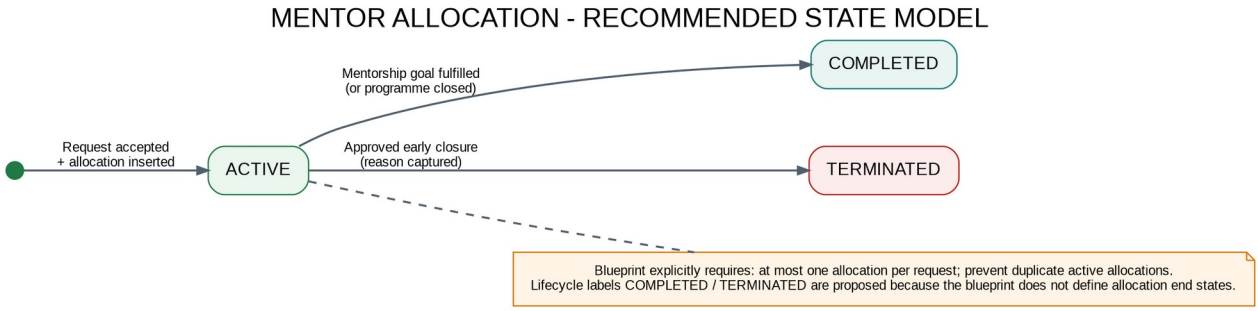


Figure 3. Proposed allocation lifecycle. ACTIVE is the immediate result of accepting a request; end states are recommended extensions.

What an allocation represents

An allocation is the durable relationship between one student and one mentor after the mentor accepts a request. It is not the request itself and it is not an individual booked session. One allocation can support multiple sessions over time.

Blueprint-defined allocation controls

Rule	Implementation implication
A request produces zero or one allocation.	Enforce UNIQUE(mentorship_request_id) in mentor_allocation.
A mentor must be eligible and within capacity.	Check capacity before inserting the allocation; serialize competing acceptances by locking the mentor profile row.
Duplicate active allocations are prevented.	Use a transaction and a uniqueness/business-key strategy; do not rely on a frontend check.
Acceptance and allocation creation are one logical operation.	Update request status, insert allocation and insert outbox event in one MySQL transaction.
Allocation supports multiple sessions.	Use allocation_id as a foreign key from mentorship_session / booking where the business model requires it.

Recommended allocation statuses (confirm with team)

Status	Meaning	Allowed outgoing transition
ACTIVE	The mentorship relationship is currently valid.	COMPLETED or TERMINATED
COMPLETED	Mentorship goal/programme has been concluded.	Terminal
TERMINATED	Relationship ended before normal completion; reason is recorded.	Terminal

Open decision

The supplied blueprint requires an allocation record and prevention of duplicate active allocations, but it does not define allocation status names or end-of-relationship rules. ACTIVE / COMPLETED / TERMINATED is a proposed state set, not a source requirement.

05 / SCHEDULING STATE MODEL

Availability and booking lifecycle

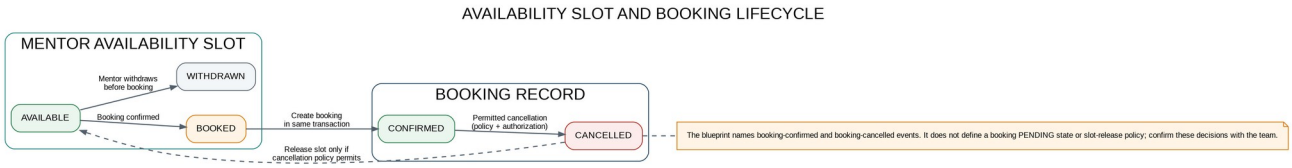


Figure 4. Slot state and booking state are related but distinct. Slot release after cancellation is a policy decision.

Distinguish the records

Record	What it means	Typical data
Mentor Availability	A time window or bookable slot offered by a mentor.	mentor_id, start_at, end_at, timezone, slot_status, created_at
Booking	A student's confirmed reservation for an offered slot.	student_id, mentor_id, allocation_id (if applicable), availability_id, purpose, booking_status
Mentorship Session	The actual offline meeting and its outcome.	booking_id, session_status, completed_at, topics, skills_assessed, observations, next_steps

Booking confirmation guards

- The mentor slot is still available at the moment of confirmation.
- The student has no overlapping confirmed appointment.
- The mentor has no overlapping confirmed appointment and has not exceeded the allowed capacity.
- The student-to-mentor request/allocation relationship is valid for the booking type.
- The submission is not a duplicate retry of the same client request.
- The slot and booking rows are updated in one atomic transaction.

Recommended status treatment

The blueprint explicitly refers to Booking Confirmed and Booking Cancelled notifications, but it does not specify a full booking status enum. A minimal design can use CONFIRMED -> CANCELLED. The slot may use AVAILABLE -> BOOKED, and optionally WITHDRAWN before booking. Whether cancellation returns a slot to AVAILABLE must be decided as a cancellation policy; do not infer it automatically.

06 / TRANSACTION AND CONCURRENCY

Atomic booking - detailed sequence

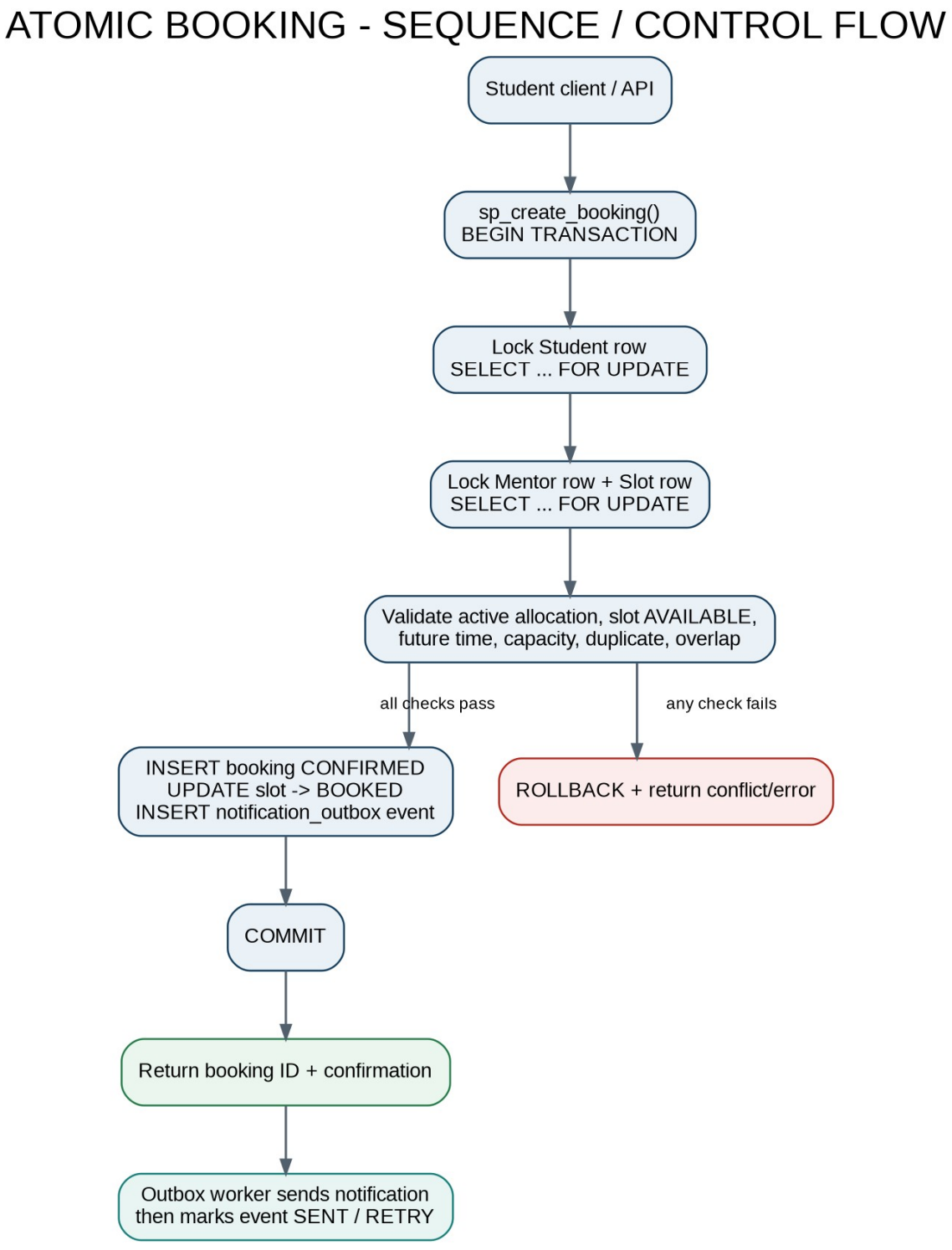


Figure 5. Booking succeeds only after all guards pass. Failure rolls back the entire write set.

Why this operation must be atomic

Consider two students selecting the same mentor slot at nearly the same time. If both read AVAILABLE before either writes, a simple application-level check can allow both to proceed. The booking routine must lock the relevant rows, re-check current state while holding those locks, and commit the slot change and booking row together.

Recommended lock order

Order	Resource locked	Reason
1	Student profile / booking guard row	Serialises booking writes for the same student so overlapping bookings cannot slip through.
2	Mentor profile / capacity guard row	Serialises capacity and overlapping-mentor checks across different slots.
3	Selected availability slot	Prevents two students from claiming the same slot.
4	Any existing overlapping booking rows	Lock and inspect the current conflict set when needed; use matching indexes.

Concurrency recommendation

Every code path that creates or changes a booking must acquire guard locks in the same order. Keep transactions short. Add a UNIQUE constraint on availability_id for active/one-time slot consumption where your schema supports it. Handle a deadlock by retrying the complete transaction, not by continuing halfway through it.

Overlap predicate

```
existing.start_at < requested_end
AND existing.end_at > requested_start
AND existing.booking_status = 'CONFIRMED'
```

This predicate detects time-range intersections. Apply it for the student and the mentor. The conflict check is only safe when performed under the agreed lock strategy; a plain SELECT followed by INSERT is not sufficient under concurrent load.

07 / ACTUAL MEETING STATE

Session lifecycle and outcome recording

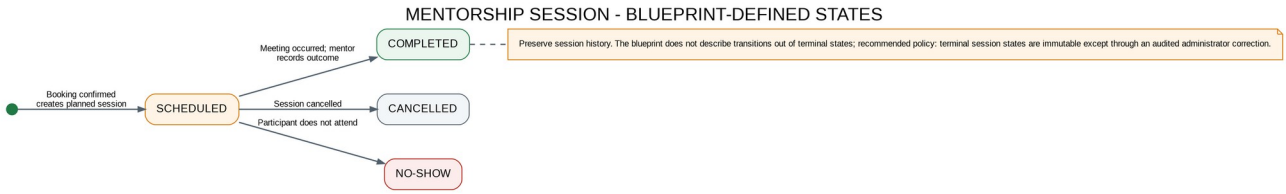


Figure 6. Blueprint-defined session states: Scheduled, Completed, Cancelled and No-show.

Session workflow

1. A confirmed booking may create one planned session record (Booking -> Session is optional one-to-one in the conceptual model).
2. Before the meeting, the session is SCHEDULED. The session is not marked completed merely because a booking exists.
3. After the offline meeting, the mentor records whether the session was completed, cancelled or the participant was a no-show.
4. For a completed session, record topics discussed, skills assessed, performance observations, improvement areas, next steps and recommended preparation.
5. Preserve session history for future progress tracking and analytics.

Session state rules

Current	Event	Next	Required validation / effect
SCHEDULED	Meeting took place; mentor records outcome	COMPLETED	Only assigned mentor / authorised administrator may record outcome; set completed_at; retain notes.
SCHEDULED	Meeting cancelled	CANCELLED	Capture who cancelled and reason/time; notify both participants.
SCHEDULED	Participant did not attend	NO_SHOW	Capture no-show actor/notes; notify relevant participants as policy requires.
COMPLETED / CANCELLED / NO_SHOW	Ordinary user update	No change	Treat as terminal; correction requires an audited admin procedure (recommended).

Do not conflate booking with session

A cancelled booking and a cancelled session are different facts. The booking captures the reservation lifecycle; the session captures the actual meeting lifecycle. Use separate statuses and timestamps.

08 / POST-SESSION RECORDS

Feedback and mentor review lifecycle

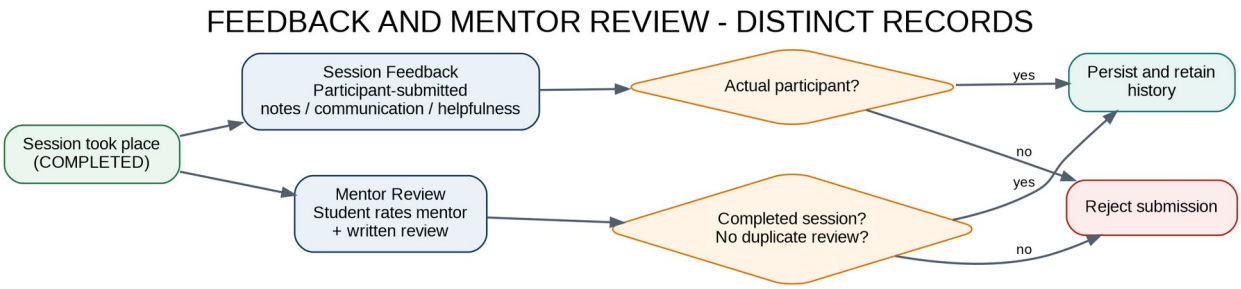


Figure 7. Feedback and rating/review are distinct business concepts.

Session feedback

The blueprint allows structured feedback from either actual participant. Example dimensions include communication, helpfulness, technical preparation and session notes. A feedback record must reference the actual session and its author.

Mentor review

A mentor review is the student's rating and written review of the mentor after a completed session. It is not simply another field inside generic feedback.

Rule	Enforcement
Feedback is not accepted before the session has taken place.	Procedure validates session state; optional trigger provides a second database guard.
Only actual participants submit session feedback.	Resolve user_id from the booking/session relationship; never trust a client-supplied participant ID.
At most one student mentor review per session.	UNIQUE(session_id, student_id) on mentor_review.
Review must reference a valid completed session.	Foreign key + procedure validation; review references session_id.
Review history is not silently overwritten.	Insert a new immutable record or use an explicitly audited correction workflow.

Suggested lifecycle

A simple model is DRAFT (optional client-side only) -> SUBMITTED (persistent). The supplied blueprint does not define a separate review moderation, edit, publication or dispute state machine. Add such states only if the team decides they are required.

09 / EVENT DELIVERY

Notifications and transactional outbox

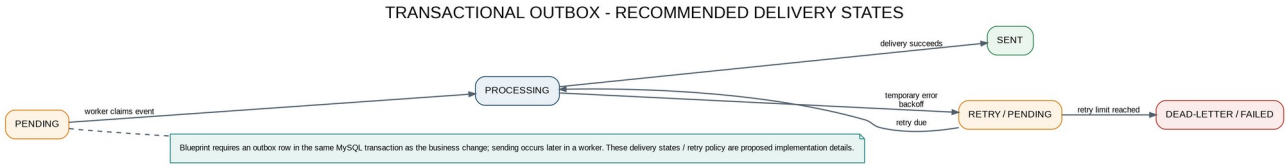


Figure 8. Outbox delivery lifecycle is a recommended operational pattern; business events are created in MySQL.

Blueprint event-to-recipient mapping

Business event	Recipient
Mentorship request submitted	Mentor
Request accepted	Student
Request rejected	Student
Booking confirmed	Both participants
Booking cancelled	Both participants
Session reminder	Both participants
Feedback submitted	Relevant participant

Outbox pattern

6. The procedure performs the business change and inserts an outbox event within the same MySQL transaction.
7. If the business transaction rolls back, the outbox event rolls back too; no false notification is delivered.
8. A separate worker polls unprocessed events, claims one safely, sends the message and marks it SENT.
9. Temporary failures are retried with backoff; permanent failures are moved to a dead-letter state for inspection.
10. Use an idempotency key / unique event key so retries do not create duplicate user-visible notifications.

Critical implementation rule

Do not send email, push notifications or call a third-party API from a database trigger. The trigger/procedure writes a durable event record; the worker performs external delivery after commit.

10 / DATABASE AUTOMATION

Trigger design for AlumniConnect

A trigger is a named database object that automatically executes in response to an INSERT, UPDATE or DELETE on a table. MySQL row triggers can run BEFORE or AFTER the row operation and use OLD and NEW values. The course material introduces triggers as a mechanism for enforcing integrity rules, monitoring updates and maintaining derived information.

Where triggers add value

Trigger	Timing / event	Purpose	Why it belongs in a trigger
trg_request_state_guard	BEFORE UPDATE on mentorship_request	Reject invalid state transitions with SIGNAL.	A database-level guard protects the table even if an unexpected client updates it.
trg_request_audit	AFTER UPDATE on mentorship_request	Write old/new status to audit_log when status changes.	Automatically records status history for all SQL updates.
trg_session_state_guard	BEFORE UPDATE on mentorship_session	Allow SCHEDULED -> COMPLETED / CANCELLED / NO_SHOW only.	Prevents terminal-state regression.
trg_session_audit	AFTER UPDATE on mentorship_session	Record outcome state changes and timestamps.	Provides an audit trail.
trg_booking_audit	AFTER INSERT / UPDATE on booking	Record confirmation and cancellation changes.	Records key reservation events.
trg_feedback_guard (optional)	BEFORE INSERT on mentor_review	Reject invalid rating values / non-completed session.	Extra integrity safeguard; enforce main business flow in a procedure too.

What should not be done in triggers

- Do not put the entire booking transaction in a trigger. Stored procedures / backend transaction orchestration are clearer for multi-row business workflows.
- Do not use triggers to send email or call HTTP services. Triggers are for database-side work.
- Do not issue COMMIT, ROLLBACK or START TRANSACTION inside a trigger. The trigger runs as part of the statement's transaction.
- Avoid duplicating the same outbox event in both a trigger and a stored procedure. Choose one owner for event creation.
- Do not rely on triggers alone to prevent concurrent slot claims; use transaction locks and unique constraints.

11 / MYSQL IMPLEMENTATION EXAMPLES

Trigger examples - state guards and audit

The following examples assume MySQL 8.x, InnoDB and the conceptual table names shown in this document. Rename columns to match the final logical schema. Keep the trigger small, deterministic and limited to database-local validation/audit work.

A. Reject an invalid mentorship-request transition

```
DELIMITER $$
CREATE TRIGGER trg_request_state_guard
BEFORE UPDATE ON mentorship_request
FOR EACH ROW
BEGIN
  IF OLD.status <> NEW.status
    AND OLD.status <> 'PENDING' THEN
     SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT = 'A decided request cannot be changed';
  END IF;

  IF OLD.status = 'PENDING'
    AND NEW.status NOT IN ('PENDING', 'ACCEPTED',
                  'REJECTED', 'CANCELLED') THEN
     SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT = 'Invalid mentorship request state';
  END IF;
END$$
DELIMITER ;
```

This guard prevents an accepted/rejected/cancelled request from being silently moved back to pending or another state. The stored procedure still checks actor identity and business eligibility because a trigger should not replace application authorization.

B. Audit a request status change

```
DELIMITER $$
CREATE TRIGGER trg_request_status_audit
AFTER UPDATE ON mentorship_request
FOR EACH ROW
BEGIN
  IF OLD.status <> NEW.status THEN
    INSERT INTO audit_log
      (entity_type, entity_id, action_name,
       old_value, new_value, changed_at)
    VALUES
      ('MENTORSHIP_REQUEST', NEW.request_id,
       'STATUS_CHANGED', OLD.status, NEW.status,
       CURRENT_TIMESTAMP);
  END IF;
END$$
DELIMITER ;
```

For production-quality auditing, extend the table with actor_user_id, request correlation ID, source module and structured JSON snapshots. The backend should set a trusted actor context for the database connection; reset it correctly when returning connections to a pool.

11B / MYSQL IMPLEMENTATION EXAMPLE

Trigger example - session lifecycle guard

C. Guard the mentorship-session lifecycle

```
DELIMITER $$
CREATE TRIGGER trg_session_state_guard
BEFORE UPDATE ON mentorship_session
FOR EACH ROW
BEGIN
  IF OLD.session_status <> NEW.session_status
    AND OLD.session_status <> 'SCHEDULED' THEN
     SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT = 'Terminal session state cannot be changed';
  END IF;

  IF OLD.session_status = 'SCHEDULED'
    AND NEW.session_status NOT IN
       ('SCHEDULED', 'COMPLETED', 'CANCELLED', 'NO_SHOW') THEN
     SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT = 'Invalid session state';
  END IF;
END$$
DELIMITER ;
```

This guard follows the lifecycle stated in the blueprint. Treat terminal outcomes as immutable in ordinary user workflows; if administrators must correct a mistaken outcome, use a separate privileged correction routine that records who changed it and why.

Trigger deployment checklist

- Create triggers through version-controlled database migrations.
- Inspect definitions using SHOW CREATE TRIGGER and keep trigger names unique and descriptive.
- Test multi-row INSERT/UPDATE statements because row triggers execute once per affected row.
- Verify that trigger writes roll back with the statement if a later operation in the same transaction fails.
- Avoid duplicate audit/outbox rows when procedures and triggers both touch the same event.

12 / DATABASE BUSINESS OPERATIONS

Stored procedures - responsibility map

A stored procedure is a named collection of SQL statements stored and executed in the database. Your Unit 2 slides cover procedure names, parameter lists and SQL bodies, with IN, OUT and INOUT parameter modes. A procedure is suitable for a multi-step operation that must apply the same rules every time.

Procedure	Input	Core responsibilities	Transaction?
sp_create_mentorship_request	student_id, mentor_id, purpose, message	Validate actor/profile and target; insert PENDING request; create request-submitted outbox event.	Yes
sp_respond_to_mentorship_request	request_id, mentor_id, decision, reason	Lock request; verify PENDING + assigned mentor; check capacity; insert allocation on ACCEPT; update request; insert outbox event.	Yes
sp_create_booking	student_id, mentor_id, slot_id, allocation_id, purpose, idempotency_key	Lock student, mentor and slot; validate eligibility and overlaps; insert booking; mark slot BOOKED; insert outbox.	Yes
sp_cancel_booking	booking_id, actor_id, reason	Lock booking; authorise participant; check cancellation policy; update status; release slot only if policy permits; outbox.	Yes
sp_record_session_outcome	session_id, mentor_id, outcome, notes...	Verify mentor/session; transition SCHEDULED to one terminal outcome; persist observations and next steps; outbox.	Yes
sp_submit_session_feedback	session_id, reviewer_id, feedback fields	Verify completed session and participant; insert feedback or review; enforce one review per session/student.	Yes
sp_get_mentor_recommendations	student_id, filters	Read candidate ranking from graph/query results; apply MySQL eligibility and capacity checks; persist recommendation run if retained.	Usually read-only + optional write

Procedure parameter modes

Mode	Meaning	AlumniConnect example
IN	Caller passes an input value.	IN p_request_id BIGINT
OUT	Procedure returns a value to the caller.	OUT p_new_booking_id BIGINT
INOUT	Caller passes a value that the procedure may modify and return.	INOUT p_page_size INT (only if there is a real use case)

Transaction error handling: Every procedure that starts a transaction should declare an EXIT HANDLER FOR SQLEXCEPTION that performs ROLLBACK and RESIGNAL. The API must not swallow the error or return a false success.

13 / TRANSACTIONAL REQUEST DECISION

Procedure example - accept or reject a request

This is a reference template, not a drop-in migration. It assumes request_id, student_id, mentor_id, status and mentor_profile.max_active_mentees. Add exact schema names, audit columns, actor authentication and business policy from your final ER-to-relational mapping.

```
DELIMITER $$
CREATE PROCEDURE sp_respond_to_mentorship_request(
    IN p_request_id BIGINT,
    IN p_mentor_id  BIGINT,
    IN p_decision   VARCHAR(10),
    IN p_reason     VARCHAR(500)
)
proc: BEGIN
    DECLARE v_found INT DEFAULT 1;
    DECLARE v_student_id BIGINT;
    DECLARE v_request_mentor_id BIGINT;
    DECLARE v_status VARCHAR(20);
    DECLARE v_capacity INT;
    DECLARE v_active_count INT;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_found = 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    IF p_decision NOT IN ('ACCEPT', 'REJECT') THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Decision must be ACCEPT or REJECT';
    END IF;

    START TRANSACTION;
    SELECT student_id, mentor_id, status
      INTO v_student_id, v_request_mentor_id, v_status
      FROM mentorship_request
     WHERE request_id = p_request_id
     FOR UPDATE;

    IF v_found = 0 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Mentorship request not found';
    END IF;
    IF v_request_mentor_id <> p_mentor_id THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Only the assigned mentor may decide';
    END IF;
    IF v_status <> 'PENDING' THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Request is no longer pending';
    END IF;

    IF p_decision = 'ACCEPT' THEN
        SET v_found = 1;
        SELECT max_active_mentees
          INTO v_capacity
          FROM mentor_profile
         WHERE mentor_id = p_mentor_id
         FOR UPDATE;
        IF v_found = 0 THEN
            SIGNAL SQLSTATE '45000'
              SET MESSAGE_TEXT = 'Mentor profile not found';
        END IF;

        SELECT COUNT(*) INTO v_active_count
          FROM mentor_allocation
         WHERE mentor_id = p_mentor_id
           AND status = 'ACTIVE';

        IF v_active_count >= v_capacity THEN
            SIGNAL SQLSTATE '45000'
              SET MESSAGE_TEXT = 'Mentor capacity reached';
        END IF;
```

13B / REMAINING TRANSACTION WRITES

Procedure example - request acceptance (continued)

```
INSERT INTO mentor_allocation
          (mentorship_request_id, student_id, mentor_id,
           status, created_at)
       VALUES
          (p_request_id, v_student_id, p_mentor_id,
           'ACTIVE', CURRENT_TIMESTAMP);

       UPDATE mentorship_request
         SET status = 'ACCEPTED', decided_at = CURRENT_TIMESTAMP,
             decision_reason = p_reason
       WHERE request_id = p_request_id;

       INSERT INTO notification_outbox
          (event_type, aggregate_type, aggregate_id, payload, status)
       VALUES
          ('REQUEST_ACCEPTED', 'MENTORSHIP_REQUEST', p_request_id,
           JSON_OBJECT('student_id', v_student_id,
                       'mentor_id', p_mentor_id), 'PENDING');
    ELSE
       UPDATE mentorship_request
         SET status = 'REJECTED', decided_at = CURRENT_TIMESTAMP,
             decision_reason = p_reason
       WHERE request_id = p_request_id;

       INSERT INTO notification_outbox
          (event_type, aggregate_type, aggregate_id, payload, status)
       VALUES
          ('REQUEST_REJECTED', 'MENTORSHIP_REQUEST', p_request_id,
           JSON_OBJECT('student_id', v_student_id,
                       'mentor_id', p_mentor_id), 'PENDING');
    END IF;

    COMMIT;
END$$
DELIMITER ;
```

What this procedure demonstrates

IN parameters, row-level locking, state validation, capacity validation, conditional INSERT/UPDATE, JSON event payloads, one transaction, exception rollback and RESIGNAL. It implements the blueprint’s requirement that request acceptance, allocation creation and notification-event recording succeed or fail together.

14 / TRANSACTIONAL SCHEDULING

Procedure example - atomic booking

The booking routine is the most concurrency-sensitive operation in AlumniConnect. The template below shows the required control sequence. The exact SQL for overlap checks must match your final slot representation and indexes.

```
-- PSEUDOCODE-LEVEL MYSQL ROUTINE TEMPLATE
-- Use one deterministic lock order in every booking/cancellation path.

START TRANSACTION;

-- 1. Serialise requests for the same student.
SELECT student_id
  FROM student_profile
 WHERE student_id = p_student_id
 FOR UPDATE;

-- 2. Serialise capacity / overlapping-mentor checks.
SELECT mentor_id, max_active_mentees
  FROM mentor_profile
 WHERE mentor_id = p_mentor_id
 FOR UPDATE;

-- 3. Lock the selected availability row.
SELECT mentor_id, start_at, end_at, status
  FROM mentor_availability
 WHERE availability_id = p_slot_id
 FOR UPDATE;

-- 4. Validate slot is still AVAILABLE and is in the future.
-- 5. Validate allocation is ACTIVE (where booking requires allocation).
-- 6. Check confirmed overlap for this student.
--    existing.start_at < requested_end
--    AND existing.end_at > requested_start
-- 7. Check confirmed overlap for this mentor.
-- 8. Validate mentor capacity and request idempotency.

INSERT INTO booking (..., booking_status, ...)
VALUES (..., 'CONFIRMED', ...);

UPDATE mentor_availability
   SET status = 'BOOKED'
 WHERE availability_id = p_slot_id
   AND status = 'AVAILABLE';

-- Check ROW_COUNT() = 1; otherwise SIGNAL and ROLLBACK.
INSERT INTO notification_outbox (...)
VALUES (..., 'BOOKING_CONFIRMED', ..., 'PENDING');

COMMIT;

-- On any SQL exception: ROLLBACK; RESIGNAL;
```

Mandatory booking constraints

- UNIQUE(availability_id) on booking if each offered slot can only be consumed once.
- Foreign keys from booking to student, mentor, availability and (where required) allocation.
- CHECK end_at > start_at; CHECK booking purpose is one of the supported types.
- Index booking(student_id, booking_status, start_at, end_at).
- Index booking(mentor_id, booking_status, start_at, end_at).
- An idempotency key with a unique constraint prevents retrying the same client submission from creating duplicate bookings.

15 / REMAINING OPERATIONS

Procedures - cancellation, session outcome and feedback

A. Cancel a booking

```
-- Transaction outline
START TRANSACTION;
SELECT student_id, mentor_id, availability_id, booking_status, start_at
  INTO ...
  FROM booking WHERE booking_id = p_booking_id FOR UPDATE;

-- Verify actor is one of the participants or authorised admin.
-- Require booking_status = 'CONFIRMED'.
-- Apply cancellation cutoff / reason policy.
UPDATE booking
   SET booking_status = 'CANCELLED',
       cancelled_at = CURRENT_TIMESTAMP,
       cancellation_reason = p_reason
 WHERE booking_id = p_booking_id;

-- Re-open the slot only when the agreed policy permits it.
-- Insert BOOKING_CANCELLED outbox event.
COMMIT;
```

B. Record session outcome

```
-- Transaction outline
START TRANSACTION;
SELECT booking_id, mentor_id, session_status
  INTO ...
  FROM mentorship_session
 WHERE session_id = p_session_id
 FOR UPDATE;

-- Verify assigned mentor (or authorised administrator).
-- Require session_status = 'SCHEDULED'.
-- Validate p_outcome in ('COMPLETED','CANCELLED','NO_SHOW').
UPDATE mentorship_session
   SET session_status = p_outcome,
       completed_at = CASE WHEN p_outcome = 'COMPLETED'
                           THEN CURRENT_TIMESTAMP ELSE NULL END,
       discussion_topics = p_topics,
       skills_assessed = p_skills,
       performance_observations = p_observations,
       improvement_areas = p_improvements,
       next_steps = p_next_steps,
       preparation_notes = p_preparation
 WHERE session_id = p_session_id;

-- Insert appropriate SESSION_* outbox event.
COMMIT;
```

C. Submit feedback / mentor review

```
START TRANSACTION;
SELECT session_status, student_id, mentor_id
  INTO ...
  FROM v_session_participants
 WHERE session_id = p_session_id;

-- Require COMPLETED and reviewer_id is an actual participant.
-- If this is a mentor review, require reviewer is the student.
-- UNIQUE(session_id, student_id) prevents duplicate review.
INSERT INTO session_feedback (...) VALUES (...);
-- Or INSERT INTO mentor_review (...) for the rating/review record.
-- Insert FEEDBACK_SUBMITTED outbox event if required.
COMMIT;
```

16 / DATA INTEGRITY

Constraints, indexes and idempotency

Integrity mechanisms by problem

Problem	Primary mechanism	Supporting mechanism
Invalid request state transition	Procedure + BEFORE UPDATE trigger guard	CHECK/ENUM domain; audit row on change.
Two allocations for one request	UNIQUE(mentorship_request_id)	Transaction locks mentor row and verifies capacity.
Same slot booked twice	UNIQUE(availability_id) or unique active-slot key	Lock availability row using SELECT ... FOR UPDATE.
Student/mentor overlapping appointments	Booking procedure under student + mentor guard locks	Overlap indexes; consistent lock ordering.
Booking for inactive relationship	Procedure verifies allocation status	Foreign key ensures referenced allocation exists.
Review before completed session	Feedback/review procedure validates state	Optional BEFORE INSERT trigger; FK + unique constraint.
Duplicate HTTP retry	Unique idempotency_key	Return the original booking result on duplicate retry.
False notification after rollback	Outbox event inserted in same transaction	Worker sends only committed outbox records.
History silently lost	No hard delete for business history	Audit log; soft archive only under explicit policy.

Suggested indexes

Table	Index / constraint	Why
mentorship_request	(mentor_id, status, created_at)	Mentor inbox and pending request list.
mentor_allocation	UNIQUE(mentorship_request_id); (mentor_id, status)	At most one allocation per request; capacity count.
mentor_availability	(mentor_id, status, start_at)	Find future open slots.
booking	UNIQUE(availability_id); (student_id, status, start_at, end_at); (mentor_id, status, start_at, end_at)	One booking per slot and efficient overlap checks.
mentorship_session	UNIQUE(booking_id); (mentor_id, status, scheduled_at)	One session per booking and mentor session list.
mentor_review	UNIQUE(session_id, student_id)	At most one student review per session.
notification_outbox	(status, available_at, created_at); UNIQUE(event_key)	Worker polling and idempotency.

17 / UNIT 4 INTEGRATION

Transactions, ACID and concurrency

ACID applied to request acceptance

Property	How AlumniConnect demonstrates it
Atomicity	Request state update + allocation insert + outbox event commit together or all roll back.
Consistency	Foreign keys, unique constraints, allowed status transitions and mentor capacity rules remain valid.
Isolation	Concurrent acceptances / bookings cannot read stale eligibility and both claim the same resource.
Durability	Once committed, the request decision, allocation, booking and outbox event persist after failure.

Locking and deadlocks

- Use SELECT ... FOR UPDATE for rows that will be changed or whose availability/capacity must be protected.
- Keep the transaction short: do not call external notification providers while holding database locks.
- Lock rows in a consistent order across procedures to reduce circular waits.
- A deadlock is a cycle of transactions waiting for each other. InnoDB detects deadlocks and rolls back a victim; the application should retry the entire transaction with bounded retries and jitter.
- Use appropriate indexes so a locking read does not scan and lock far more rows than needed.

Suggested transaction boundaries

Operation	One transaction must include
Accept request	Lock request; validate mentor; lock mentor capacity; insert allocation; update request; insert outbox event.
Create booking	Lock student, mentor and availability; validate overlaps and capacity; insert booking; change slot; insert outbox event.
Cancel booking	Lock booking; authorise actor; update booking; apply slot-release rule; insert outbox event.
Record outcome	Lock session; validate SCHEDULED; write outcome and notes; insert outbox event.
Submit feedback	Lock / validate session; validate participant and review uniqueness; insert feedback/review; insert outbox event.

18 / UNIT 2 AND QUERY DESIGN

Additional DBMS concepts worth demonstrating

The workflow database should also provide meaningful examples for the rest of the supplied Unit 2 syllabus. These are not additional state machines; they are reporting, data-integrity and reuse tools over the workflow records.

DBMS concept	AlumniConnect use case	Example
Views	Expose a safe public mentor directory without private identity/contact data.	v_public_mentor_directory
Stored functions	Compute a reusable metric from profile data.	fn_profile_completeness(student_id)
CTEs	Analyse progression from request to actual mentoring outcomes.	WITH request_funnel AS (...) SELECT submitted, accepted, allocated, completed
Window functions	Rank mentors inside each mentorship domain.	ROW_NUMBER() OVER (PARTITION BY domain_id ORDER BY score DESC)
Aggregates + HAVING	Identify highly rated active mentors with sufficient completed sessions.	AVG(rating), COUNT(completed sessions), HAVING ...
Nested / correlated queries	Find students with no allocation; find mentors with above-domain-average experience.	NOT EXISTS / correlated subquery
Full-text search	Search mentor biographies and professional experience descriptions.	MATCH(bio, experience_summary) AGAINST (...)
Triggers	Audit status changes and reject invalid transition paths.	BEFORE UPDATE guard + AFTER UPDATE audit trigger
Stored procedures	Process request acceptance and atomic booking.	sp_respond_to_mentorship_request; sp_create_booking
EXPLAIN / indexing	Show a before-and-after query plan for mentor discovery and history pages.	EXPLAIN SELECT ...

Recommended analytical views

- v_pending_requests_by_mentor - mentor inbox and pending workload.
- v_mentor_capacity - active allocation count versus configured capacity.
- v_booking_calendar - confirmed future bookings with participant names and slot times.
- v_session_history - completed/cancelled/no-show outcome history for each allocation.
- v_mentor_rating_summary - average rating, review count and completed-session count.
- v_request_to_session_funnel - request -> accepted -> allocated -> booked -> completed.

19 / VERIFICATION PLAN

Testing the lifecycle and transitions

Test state transitions and concurrency at the database boundary, not only through the UI. Use two independent database sessions for simultaneous booking/acceptance scenarios.

Test ID	Scenario	Expected result
WF-01	Student submits a valid mentorship request.	One PENDING request; one REQUEST_SUBMITTED outbox event.
WF-02	Mentor accepts a PENDING request with remaining capacity.	Request ACCEPTED; one ACTIVE allocation; one outbox event; all committed together.
WF-03	Mentor accepts a request that is already REJECTED.	Operation rejected; no allocation inserted.
WF-04	Two concurrent accepts compete for the last mentor capacity.	Only one may allocate; second re-checks capacity and fails cleanly.
WF-05	Same request acceptance API call is retried.	No duplicate allocation; return original decision / conflict deterministically.
WF-06	Two students book the same slot simultaneously.	Exactly one CONFIRMED booking; slot marked BOOKED once.
WF-07	One student books two overlapping slots concurrently.	Only one confirmed appointment survives.
WF-08	Mentor has overlapping confirmed slots.	Conflicting booking rejected.
WF-09	Booking insert succeeds but outbox insert fails.	Entire transaction rolls back; slot remains AVAILABLE.
WF-10	Cancel a confirmed booking.	Booking CANCELLED; both participants notified; slot release follows explicit policy.
WF-11	Mentor marks a scheduled session COMPLETED.	Outcome and notes saved; session history retained.
WF-12	Attempt to move COMPLETED session back to SCHEDULED.	Trigger/procedure rejects invalid transition.
WF-13	Non-participant submits session feedback.	Rejected; no feedback row.
WF-14	Student reviews same session twice.	Second review rejected by unique constraint / procedure.
WF-15	Outbox provider temporarily fails.	Event moves to retry state; user-visible duplicate is avoided on retry.
WF-16	Deadlock or lock timeout occurs.	One transaction rolls back; API reports retryable conflict; full operation can be retried.

20 / BUILD PLAN

Recommended implementation order and open decisions

Suggested implementation order

11. Finalise the logical schema: table names, keys, foreign keys, status domains, timestamp columns and unique constraints.
12. Implement request submission and mentor accept/reject procedures; test allocation capacity and duplicate prevention.
13. Implement mentor availability and the booking transaction; test same-slot and overlapping-booking concurrency with two SQL sessions.
14. Implement session creation and outcome recording; keep booking and session state separate.
15. Implement feedback and review validation; add the unique review rule.
16. Add audit triggers and transition guards after procedure flows are stable.
17. Add notification_outbox writes inside each business transaction; implement an independent worker with retry/idempotency.
18. Add views/functions/CTEs/window reports and query-plan demonstrations for Unit 2 and Unit 3.
19. Add Redis cache invalidation and Neo4j projection updates from committed outbox/domain events; never allow either to override MySQL.

Open decisions the team should record

Decision	Why it matters	Recommended default for first release
Can a student cancel after acceptance?	Blueprint defines student cancellation while request is pending; no post-acceptance withdrawal policy.	Do not change accepted request. Add an audited allocation termination path if needed.
Allocation completion / termination policy	Blueprint does not define allocation states.	Use ACTIVE / COMPLETED / TERMINATED only after team agreement.
Booking cancellation cutoff and slot release	Blueprint does not specify whether a cancelled slot becomes bookable again.	Allow release only for future slots and according to a defined cutoff.
Booking confirmation style	Blueprint talks about confirmed booking; no pending mentor approval is described.	Use immediate CONFIRMED booking after all validations pass.
Can one booking create multiple sessions?	Conceptual model states one booking to zero or one session.	Enforce UNIQUE(booking_id); create separate bookings for separate appointments.
Who may mark a session cancelled / no-show?	Blueprint describes session recording after offline meeting but not role matrix for every terminal outcome.	Mentor or authorised administrator, with actor and reason captured.
Feedback edit / moderation	Blueprint says review history should not be silently overwritten.	Append a correction record or audited admin correction; do not overwrite silently.
Outbox retry / dead-letter policy	Blueprint requires outbox but not retry configuration.	Exponential backoff, retry limit, dead-letter queue and idempotency key.

21 / REFERENCES

Source alignment and technical references

Supplied project and course material

Reference	Used for
Alumni-Student Mentorship Platform: DBMS Project Blueprint, Section 7 (Important business workflows), approx. pp. 22-27.	Request path and states; mentor acceptance; atomic booking checks; offline session states and captured notes; feedback/review rules; notification events and transactional outbox.
Alumni-Student Mentorship Platform: DBMS Project Blueprint, Sections 2-3 and 8.	Modular monolith; MySQL source of truth; Redis and Neo4j responsibilities; Unit 1-4 mapping; stored procedures for booking/acceptance; triggers for audit.
DBMS UNIT 2 course slides, topics Triggers and Stored Procedures.	Trigger event/timing, BEFORE/AFTER, INSERT/UPDATE/DELETE, FOR EACH ROW, OLD/NEW, SIGNAL SQLSTATE; stored procedure definition and IN/OUT/INOUT parameters.
DBMS Unit 4 course slides, topics Transactions, ACID, Lock-Based Protocols and Deadlock.	Atomicity, consistency, isolation, durability; concurrent transactions; lock compatibility; deadlock and rollback.
DBMS Unit 1 / Unit 3 course materials.	Relational integrity foundations, normalisation and query optimisation context.

MySQL 8.4 Reference Manual

- MySQL 8.4 Reference Manual, “Using Triggers” and “Trigger Syntax and Examples”: https://dev.mysql.com/doc/refman/8.4/en/triggers.html and https://dev.mysql.com/doc/refman/8.4/en/trigger-syntax.html
- MySQL 8.4 Reference Manual, “START TRANSACTION, COMMIT, and ROLLBACK”: https://dev.mysql.com/doc/refman/8.4/en/commit.html
- MySQL 8.4 Reference Manual, “InnoDB Locking Reads”: https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html
- MySQL 8.4 Reference Manual, “InnoDB Transaction Isolation Levels”: https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-isolation-levels.html

Important implementation notes

1. The sample SQL is illustrative and assumes the entity/column names shown. Validate it against your final relational schema before execution. 2. The blueprint is the authority for business scope; proposed allocation, booking-slot and outbox state details are clearly labelled as recommendations. 3. Database triggers and procedures should be tested with concurrent sessions, failure injection and rollback checks before the team treats them as complete.

Final design principle

Every important lifecycle transition should have one authoritative write path, one explicit guard, a durable history record and (where a notification is required) an outbox event committed in the same transaction.